

ThunderERP

Git & GitHub Engineering Reference

Team Workflow • Version Control • CI/CD • DevOps

Intellectual Property of AB Global Traders

Stack: Git • GitHub • GitHub Actions • Docker • GitHub Secrets

0. How to Use This Document

This is the official operating manual for version control, collaboration, CI/CD, and DevOps practices in ThunderERP. It is NOT a quick cheat sheet. It is a long-term reference designed to grow with your career from intern to senior engineer.



MANDATORY: Every new team member must read this document before their first commit. Violations of the workflow defined here can corrupt financial data, break inventory records, corrupt audit trails, or bring down the entire production ERP system.

Layered Reading Plan

- First pass — Day 1: Git vs GitHub, installation, account setup, and mental model.
- Second pass — Week 1: Daily workflow, commits, and pull requests.
- Third pass — Month 1+: Complete Git + CI Flow, conflict resolution, and CI/CD debugging.
- Ongoing: Open this document when you face failures, conflicts, code reviews, deployments, or disasters.



BOOKMARK THESE: Section 2 (Setup Checklist), Section 7 (The Complete Git + CI Flow), Section 12 (Debugging Playbook), and Section 21 (Cheat Sheet). You will return to these every week.

1. Git vs GitHub — The Fundamental Difference

These two words are often used interchangeably but they are completely different things. Understanding this distinction is the foundation of everything else.

Concept	Git	GitHub
What it is	Version control system (free, open-source software)	Cloud hosting platform for Git repositories
Where it runs	Locally on your laptop — entirely offline capable	In the cloud — accessible from any browser
Who made it	Linus Torvalds (creator of Linux) in 2005	Private company, acquired by Microsoft in 2018
What it does	Records every change, manages branches, history, merges	Enables team collaboration, PRs, Issues, Actions (CI/CD), Releases
Can it work alone?	Yes — git works with no internet connection	No — GitHub needs Git to function

 **SIMPLE ANALOGY:** Git is your personal time machine for code — it runs on your laptop and remembers every version of every file you have ever worked on. GitHub is the shared online garage where the whole team reviews, tests, merges, and deploys that code safely.

1.1 Why Git + CI/CD Discipline Is Non-Negotiable in ThunderERP

ThunderERP handles real money, live inventory, customer data, and complex business logic across 35+ concurrent developers. One bad merge, one skipped CI check, or one direct push to main can:

- Break stock levels causing overselling
- Corrupt financial records or audit trails
- Fail Prisma migrations on production — making the entire system unusable
- Expose security vulnerabilities in the auth module
- Overwrite another developer's work and lose days of progress

Git + CI/CD together form the safety backbone of the entire ThunderERP system. The rules in this document are not bureaucracy — they are engineering discipline that protects real data and real people.

1.2 The Three-Layer Safety System

Layer	Tool	What It Protects Against
Version Control	Git + branches	Lost work, overwrites, untracked changes
Code Review	GitHub Pull Requests	Bugs, bad logic, ERP rule violations, security holes
Automated Checks	GitHub Actions (CI/CD)	Broken builds, failing tests, schema errors, lint violations

2. Installation & Account Setup

2.1 Install Git

Windows

1. Go to <https://git-scm.com/download/windows>
2. Download and run the installer.
3. During installation: select "Git from the command line and also from 3rd-party software". Leave all other options as default.
4. Restart your terminal after installation.

macOS

```
# Option A — via Homebrew (recommended):  
brew install git  
  
# Option B — via Xcode Command Line Tools:  
xcode-select --install  
# Git is included in the tools that install.
```

Linux (Ubuntu/Debian)

```
sudo apt update  
sudo apt install git
```

Verify Installation

```
git --version  
# Should show: git version 2.40.x or higher
```

2.2 Install GitHub Desktop (Highly Recommended for Interns)

GitHub Desktop provides a visual interface for Git operations. It is especially useful when resolving merge conflicts or reviewing diffs. Download from <https://desktop.github.com>.

 **TIP:** GitHub Desktop and command-line Git work on the same repositories simultaneously. You can use Desktop for visual operations (diffs, conflict resolution) and the terminal for commands that Desktop does not expose.

2.3 Configure Git Globally

This configuration tells Git who you are. Every commit you make is tagged with this name and email permanently. Use your company email.

```
# Set your identity (mandatory — do this first):
```

```
git config --global user.name "Your Full Name"
git config --global user.email "yourname@thundererp.com"

# Set default editor for commit messages:
git config --global core.editor "code --wait"    # VS Code
# OR:
git config --global core.editor "nano"           # Nano (simpler)

# Set default branch name to "main":
git config --global init.defaultBranch main

# Enable colored terminal output:
git config --global color.ui auto

# Prevent line-ending issues between Windows and Mac/Linux:
git config --global core.autocrlf input    # Mac/Linux
git config --global core.autocrlf true     # Windows

# Verify your config:
git config --list
```

2.4 Set Up SSH Key (Secure Authentication)

SSH keys allow you to push to GitHub without entering your password every time. They are also more secure than HTTPS with passwords.

```
# Step 1: Generate an SSH key pair:
ssh-keygen -t ed25519 -C "yourname@thundererp.com"
# Press Enter to accept default location (~/.ssh/id_ed25519)
# Set a passphrase (recommended)

# Step 2: Start the SSH agent:
eval "$(ssh-agent -s)"

# Step 3: Add your key to the agent:
ssh-add ~/.ssh/id_ed25519

# Step 4: Copy your PUBLIC key to clipboard:
cat ~/.ssh/id_ed25519.pub
# Copy the entire output

# Step 5: Add to GitHub:
# → GitHub → Settings → SSH and GPG keys → New SSH key
# Paste your key. Title: "Work Laptop" or similar.

# Step 6: Test the connection:
ssh -T git@github.com
# Should show: "Hi username! You have successfully authenticated."
```

2.5 GitHub Account Setup Checklist

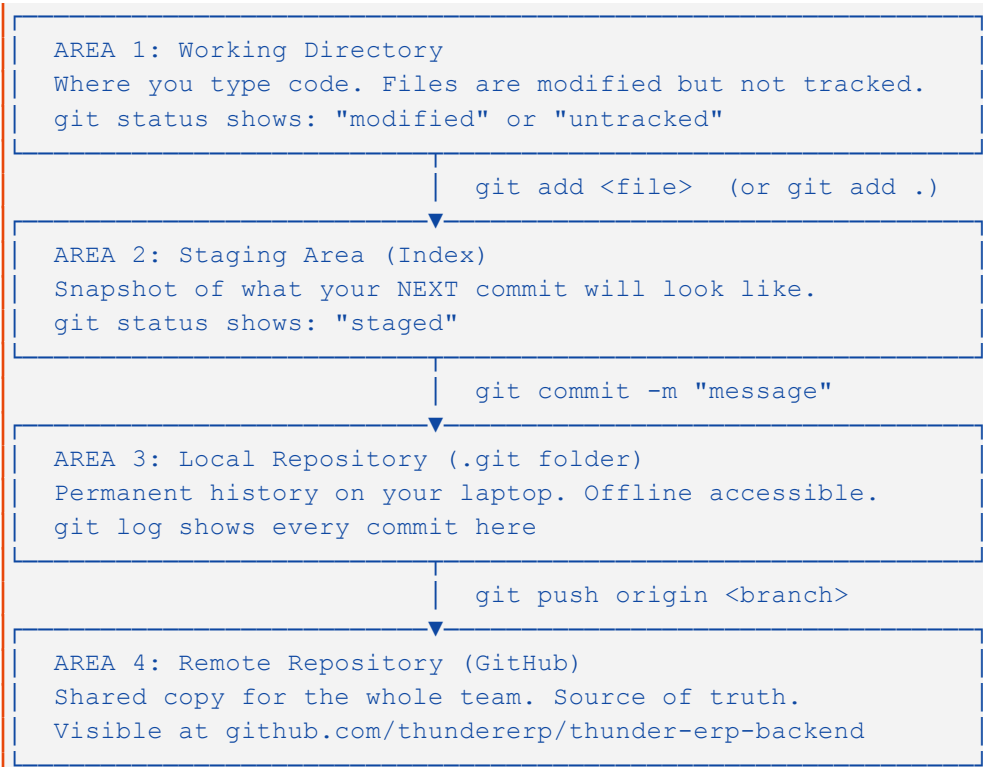
5. Create a GitHub account with your company email address.
6. Enable Two-Factor Authentication (2FA) — mandatory for the organization.
7. Send your GitHub username to the Tech Lead to receive an organization invitation.
8. Accept the ThunderERP GitHub Organization invitation from your email.
9. Verify you can see the thunder-erp-backend and thunder-erp-frontend repositories.
10. Clone the repository using SSH (see Section 3.3).
11. Confirm you CANNOT push directly to main or dev — you should get a permission error.

 **SECURITY:** Branch Protection Rules are active. Direct pushes to dev and main are blocked for everyone including senior developers. ALL changes must go through a Pull Request with passing CI checks.

3. The Mental Model — How Git Actually Works

Before you run any Git commands, you need to understand the four areas where your code can live. Confusion about these four areas is the root cause of most Git mistakes.

3.1 The Four Areas of Git



Area	Command to Move Code In	Command to Move Code Out
Working → Staging	git add <file> or git add .	git restore --staged <file>
Staging → Local Repo	git commit -m "message"	git reset --soft HEAD~1
Local Repo → Remote	git push origin <branch>	git pull origin <branch>
Remote → Local Repo	git fetch origin	git push (do not go backwards normally)

💡 **MOST IMPORTANT COMMAND:** "git status" — Run it constantly. It tells you exactly which area your files are in, what is staged, what is modified, and what branch you are on. When confused, run git status first.

3.2 What Is a Commit?

A commit is a permanent snapshot of your code at a specific point in time. Every commit has a unique SHA hash (like 3a7f2b1), an author, a timestamp, and a message. Commits are the atomic unit of version control — they are what Git tracks, what GitHub displays, and what CI/CD builds from.

```
# Each commit stores:
- A unique SHA hash:      3a7f2b1c9e4d5f6a... (40 characters)
- Author name and email:  "Jane Dev <jane@thundererp.com>"
- Timestamp:              "Mon Jan 15 10:30:00 2024 +0530"
- Commit message:         "feat(inventory): add GST calculation to product
service"
- Parent commit:          The SHA of the previous commit
- Tree (snapshot):        The complete state of all tracked files

# This chain of commits is the history of the project.
# It can never be changed – only appended to (safely).
```

3.3 Cloning the Repository

```
# Clone using SSH (recommended – requires SSH key setup from Section 2.4):
git clone git@github.com:thundererp/thunder-erp-backend.git
git clone git@github.com:thundererp/thunder-erp-frontend.git

# Enter the project:
cd thunder-erp-backend

# Verify remote connections:
git remote -v
# Should show:
# origin  git@github.com:thundererp/thunder-erp-backend.git (fetch)
# origin  git@github.com:thundererp/thunder-erp-backend.git (push)

# Verify you are on dev:
git branch
# Should show: * dev
```

4. Essential Git Commands — The Complete Reference

4.1 Repository & Status

```
# Check status of your working directory:
git status

# See commit history:
git log                # Full log with all details
git log --oneline      # Compact: one commit per line
git log --oneline --graph  # Visual branch tree
git log --oneline -10    # Last 10 commits only
git log --author="Jane" --oneline  # Commits by specific author

# See what changed in a specific commit:
git show 3a7f2b1

# See differences:
git diff                # Changes in working dir (not staged)
git diff --staged       # Changes that ARE staged
git diff main..dev      # Differences between two branches

# Every action you have ever taken (your life-saver):
git reflog
```

4.2 Branching

```
# List all local branches:
git branch

# List all branches including remote:
git branch -a

# Create a new branch from current branch:
git checkout -b feature/inventory-gst-calculation

# Create branch from specific branch (ALWAYS create from dev):
git checkout dev
git pull origin dev          # ALWAYS pull first!
git checkout -b feature/sales-order-status-workflow

# Switch to existing branch:
git checkout dev
git switch dev               # Modern equivalent of checkout

# Rename current branch:
git branch -m feature/old-name feature/new-name
```

```
# Delete local branch (safe – only if merged):
git branch -d feature/my-feature

# Force delete (unmerged branch):
git branch -D feature/abandoned-experiment

# Delete remote branch:
git push origin --delete feature/my-feature

# See which branch each commit is on:
git log --oneline --decorate
```

4.3 Staging & Committing

```
# Stage specific file:
git add src/modules/inventory/inventory.service.ts

# Stage all changes in current directory:
git add .

# Stage parts of a file interactively (advanced – very useful):
git add -p src/modules/inventory/inventory.service.ts
# Git shows each change chunk and asks: stage this? (y/n/s/q)

# Unstage a file (keep changes in working dir):
git restore --staged src/modules/inventory/inventory.service.ts

# Commit with a message:
git commit -m "feat(inventory): add GST calculation service method"

# Commit and open editor for longer message:
git commit

# Amend last commit (fix message or add forgotten file):
# WARNING: Only use BEFORE pushing – never amend a pushed commit
git commit --amend -m "feat(inventory): add GST calculation with test"

# Discard changes in working directory (IRREVERSIBLE):
git restore src/modules/inventory/inventory.service.ts
```

4.4 Pushing & Pulling

```
# Push your branch to remote for the first time:
git push -u origin feature/inventory-gst-calculation
# -u sets upstream – after this, just use: git push

# Push subsequent commits:
git push
# or explicitly:
```

```
git push origin feature/inventory-gst-calculation

# Pull latest changes from remote:
git pull origin dev

# Fetch without merging (safe — just updates remote tracking):
git fetch origin

# See what is on remote without pulling:
git fetch origin
git log --oneline origin/dev

# Pull with rebase (cleaner history — advanced):
git pull --rebase origin dev
```

4.5 Stashing — Saving Work Temporarily

Stash is your emergency parachute. If you need to switch branches urgently but have uncommitted work, stash it.

```
# Save current changes to stash:
git stash
# or with a description:
git stash push -m "WIP: half-done GST calculation"

# List all stashes:
git stash list
# Output: stash@{0}: WIP: half-done GST calculation

# Apply most recent stash (keeps it in stash list):
git stash apply

# Apply and remove from stash list:
git stash pop

# Apply a specific stash:
git stash apply stash@{2}

# Delete a stash:
git stash drop stash@{0}

# Delete all stashes:
git stash clear

# See what is in a stash before applying:
git stash show -p stash@{0}
```

4.6 Undoing Mistakes

Git has multiple levels of "undo". Choosing the wrong one can make things worse. Use this table carefully.

Situation	Command & When It Is Safe
Undo unstaged changes in a file	<code>git restore <file></code> — Safe. Irreversible for that file only.
Unstage a staged file	<code>git restore --staged <file></code> — Safe. Changes stay in working dir.
Undo last commit, keep changes staged	<code>git reset --soft HEAD~1</code> — Safe ONLY before pushing.
Undo last commit, keep changes unstaged	<code>git reset --mixed HEAD~1</code> — Safe ONLY before pushing.
Undo last commit, discard all changes	<code>git reset --hard HEAD~1</code> — DANGEROUS. Changes are gone.
Safely undo a pushed commit (shared branch)	<code>git revert <commit-sha></code> — ALWAYS use this on dev/main.
Recover a lost commit	<code>git reflog</code> → find sha → <code>git cherry-pick <sha></code>
Reset local branch to match remote exactly	<code>git reset --hard origin/dev</code> — DANGEROUS. Loses local commits.

 **CRITICAL RULE:** NEVER use `git reset --hard` or `git push --force` on dev or main. These rewrite shared history and destroy other developers' commits permanently. On shared branches, always use `git revert` instead.

4.7 Git Aliases — Work Faster

```
# Add these shortcuts to your git config:
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg "log --oneline --graph --decorate --all"
git config --global alias.unstage "restore --staged"
git config --global alias.last "log -1 HEAD"

# Usage:
git st      # instead of: git status
git lg      # beautiful branch graph
git last    # see the last commit
```

5. The .gitignore File — Never Commit These

The .gitignore file tells Git which files to never track. Committing the wrong files can expose secrets, bloat the repository with binary files, or cause "works on my machine" conflicts across the team.

5.1 ThunderERP .gitignore

```
# — Node.js —————
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# — Build Output —————
dist/
build/
.next/

# — Environment & Secrets (CRITICAL) —————
.env
.env.*
!.env.example      # Keep the TEMPLATE (no real values)

# — Prisma —————
# DO commit: prisma/schema.prisma and prisma/migrations/
# Do NOT commit generated client:
prisma/generated/

# — Logs —————
logs/
*.log

# — OS & Editor Files —————
.DS_Store
Thumbs.db
.vscode/*
!.vscode/extensions.json    # Share recommended extensions
!.vscode/settings.json      # Share project settings
.idea/

# — Test Coverage —————
coverage/
.nyc_output/

# — Docker —————
# DO commit: docker-compose.yml
# Do NOT commit:
docker-compose.override.yml
```

 **NEVER COMMIT SECRETS:** If a .env file or API key is accidentally committed, do NOT just delete it in the next commit — it is still in the git history. You must rotate the secret immediately and notify the Tech Lead. Use "git filter-branch" or BFG Repo Cleaner to purge it from history.

5.2 The .env.example File — Always Commit This

The .env.example file is the template that shows what environment variables are needed, without revealing the real values. Every developer uses this to set up their own .env file.

```
# .env.example - commit this to git:
DATABASE_URL="postgresql://USER:PASSWORD@HOST:PORT/DBNAME"
JWT_SECRET="replace-with-a-strong-random-secret"
JWT_EXPIRES_IN="24h"
VITE_API_URL="http://localhost:3000"
SMTP_HOST="smtp.example.com"
SMTP_PORT="587"

# .env - NEVER commit this:
DATABASE_URL="postgresql://admin:ActualPassword123@localhost:5432/thunder_erp"
JWT_SECRET="h3v8x2nq9pL7mR4..."
```

6. ThunderERP Branching Strategy

ThunderERP uses a structured three-tier branching strategy. Every developer works in their own feature branch and merges through pull requests. No one ever commits directly to dev or main.

6.1 The Three Permanent Branches

Branch	Purpose, Rules & Who Can Merge
main	Production. Exactly what is running in front of users. LOCKED — only Tech Leads can merge here, and only from dev after staging passes.
dev	Integration branch. All completed features merge here first. Should always be stable and passing CI. LOCKED — only via PR with passing CI.
feature/*	Your personal workspace. Created from dev, merged back to dev via PR. Short-lived — deleted after merge.

6.2 Branch Naming Convention

Branch names must be lowercase, use hyphens (no underscores or spaces), and include the module name. They must exactly match a backend module name.

```
# Format:
<type>/<module>-<short-description>

# Feature branches:
feature/inventory-gst-calculation
feature/sales-order-status-workflow
feature/crm-lead-pipeline-ui
feature/finance-invoice-payment-modal
feature/auth-refresh-token

# Bug fix branches:
fix/inventory-stock-decrement-race-condition
fix/auth-jwt-expiry-not-clearing-session

# Hotfix branches (urgent production fixes):
hotfix/sales-order-total-calculation-wrong
hotfix/auth-password-reset-broken

# Refactor branches:
refactor/inventory-service-extract-gst-helper

# Documentation branches:
docs/api-swagger-update-product-endpoints

# BAD names (never use):
```

```
my-branch  
fix  
test123  
johns-work  
update
```

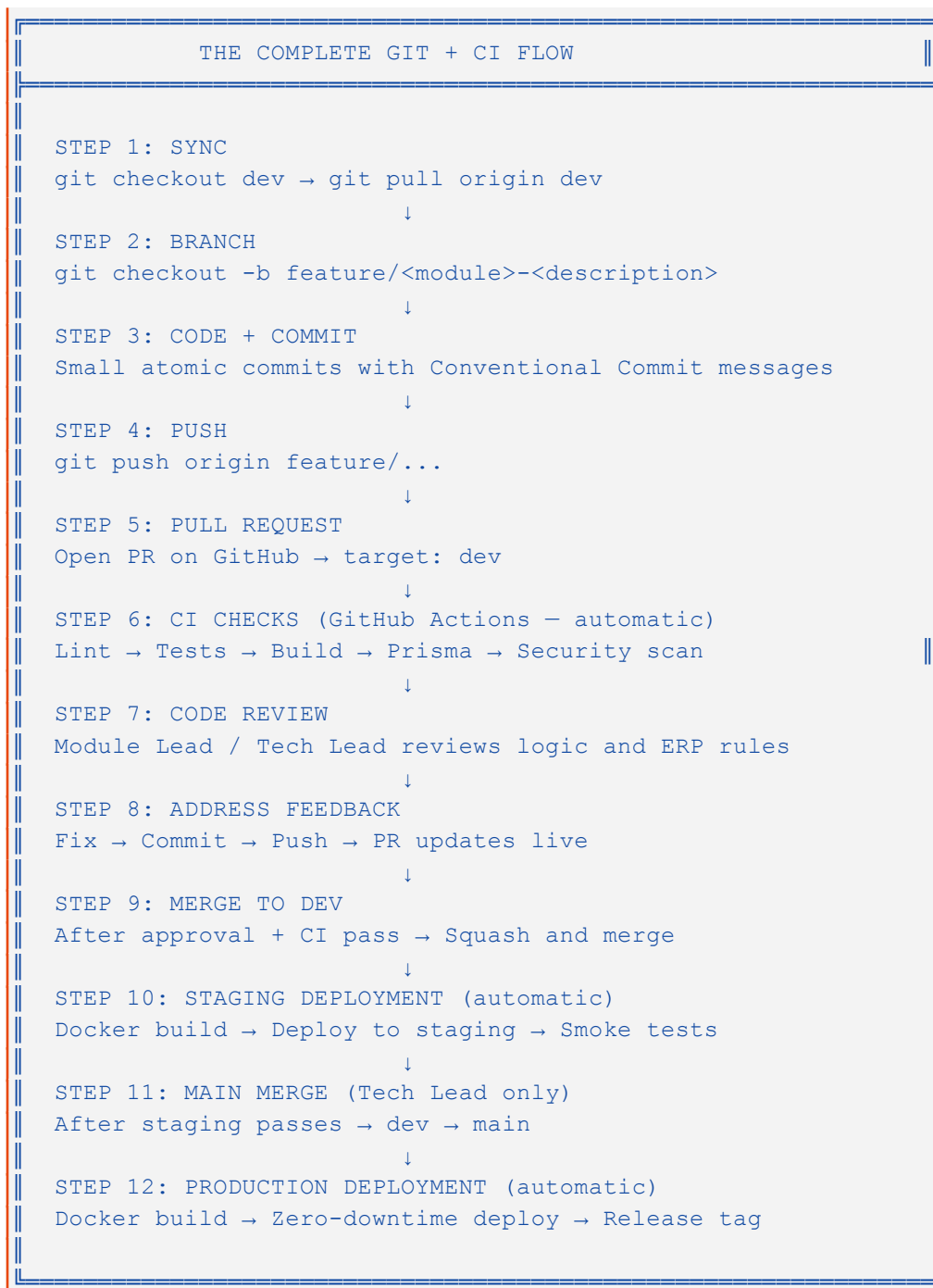
6.3 Branch Lifecycle

```
# 1. Always start from the latest dev:  
git checkout dev  
git pull origin dev  
  
# 2. Create your feature branch:  
git checkout -b feature/inventory-gst-calculation  
  
# 3. Work and commit (see Section 8 for commit rules)  
  
# 4. Keep your branch up to date with dev (do this daily):  
git fetch origin  
git rebase origin/dev  
# OR (merge approach):  
git merge origin/dev  
  
# 5. Push and open PR:  
git push -u origin feature/inventory-gst-calculation  
  
# 6. After PR is merged, clean up:  
git checkout dev  
git pull origin dev  
git branch -d feature/inventory-gst-calculation
```

 **RULE:** Feature branches must never live longer than 2–3 days without a PR. Long-lived branches accumulate conflicts and make merging painful for everyone. If a feature takes longer, break it into smaller PRs.

7. The Complete Git + CI Flow — The Golden Path

This is the mandatory end-to-end flow every developer must follow for every single change. Memorize it. This flow ensures main is always deployable and the ERP is always protected.



7.1 Key Rules of the Flow

Rule	Why It Matters
CI must pass BEFORE any human review begins	Reviewers should focus on logic, not lint errors or broken builds.
If CI fails → fix it immediately, do not ask for review	A failed CI is broken code. It must be fixed before it wastes anyone's review time.
Only Tech Leads merge to main	Production deploys are gated. No accidental or unauthorized production releases.
Every merge to dev triggers staging deployment	Every feature is validated in a production-like environment before reaching real users.
Every merge to main creates a release tag	Every production deployment is versioned and traceable. Rollbacks are fast.
PRs must have passing CI AND at least one approval	Both automated and human checks are required. Neither alone is sufficient.

8. Commit Discipline — Conventional Commits

Every commit in ThunderERP must follow the Conventional Commits specification. This is not a suggestion — it is enforced by a commit-lint hook in CI. A commit message that does not follow the format will fail the pipeline.

8.1 The Commit Message Format

```
<type>(<scope>): <short description>

[optional longer body — explain WHY, not WHAT]

[optional footer — e.g., Closes #123]

# Examples:
feat(inventory): add GST calculation to product creation service
fix(auth): resolve JWT token not clearing on logout
refactor(sales): extract order total calculation into helper function
docs(prisma): add comment explaining soft delete pattern on Product model
test(inventory): add unit tests for GST calculation edge cases
chore(deps): upgrade Prisma from 5.8 to 5.10

# With body and footer:
feat(finance): add partial payment recording to invoice

Previously invoices could only be marked as fully paid.
This adds support for recording partial payments with
automatic status update: UNPAID → PARTIALLY_PAID → PAID.

Closes #247
```

8.2 Commit Types Reference

Type	When to Use & Examples
feat	A new feature or capability. "feat(crm): add lead source tracking"
fix	A bug fix. "fix(inventory): prevent stock going negative on concurrent orders"
refactor	Code restructure with no behavior change. "refactor(auth): extract token validation to helper"
docs	Documentation only. "docs(api): add JSDoc to all InventoryService methods"
test	Adding or fixing tests. "test(sales): add integration test for order creation transaction"
chore	Build, config, tooling changes. "chore(deps): update NestJS to v10.3"
style	Formatting only (no logic change). "style(inventory): fix import ordering and spacing"

perf	Performance improvement. "perf(products): add index on tenantId for faster list queries"
ci	CI/CD configuration. "ci: add Prisma validation step to PR workflow"
revert	Reverting a previous commit. "revert: feat(sales): add bulk order import"
BREAKING CHANGE	In the footer — signals API or DB schema break. Triggers major version bump.

8.3 The Atomic Commit Principle

An atomic commit does exactly ONE thing. It passes all tests by itself. It can be reverted independently without breaking other things. This is the most important habit to build as an engineer.

BAD COMMIT: "feat(inventory): add product, fix auth bug, update readme, refactor sales service" — This is four changes in one commit. If the auth fix is bad, you cannot revert just that change.

GOOD COMMITS: "feat(inventory): add product creation endpoint" + "feat(inventory): add GST validation to product DTO" + "feat(inventory): add inventory record on product creation" — Three separate, independent commits.

8.4 Commit Message Quality Rules

- Subject line: maximum 72 characters. Enough to read in git log --oneline.
- Use imperative mood: "add feature" not "added feature" or "adding feature".
- Do NOT end the subject line with a period.
- Explain WHY in the body, not WHAT (the code shows what — the message explains why).
- Reference issue numbers in the footer: "Closes #247" or "Relates to #198".

9. Daily Developer Lifecycle — The 8-Step Loop

Every single working day follows this loop. No exceptions. Build this as a muscle memory habit.

9.1 The Daily Workflow

STEP 1: SYNC — Always start here

```
git checkout dev
```

```
git pull origin dev
```

STEP 2: BRANCH — Never work on dev directly

```
git checkout -b feature/inventory-gst-calculation
```

STEP 3: WORK — Make changes in small, focused increments

Write code → Run tests locally → Verify it works

STEP 4: COMMIT — Atomic commits with Conventional format

```
git add src/modules/inventory/inventory.service.ts
```

```
git commit -m "feat(inventory): add GST calculation"
```

STEP 5: PUSH — Send to GitHub

```
git push origin feature/inventory-gst-calculation
```

STEP 6: PULL REQUEST — Open PR on GitHub

Target: dev. Fill in description. Self-review first.

STEP 7: RESPOND — Fix CI failures and review comments

```
git add . → git commit → git push (PR updates live)
```

STEP 8: CLEANUP — After merge, delete the branch

```
git checkout dev → git pull origin dev
```

```
git branch -d feature/inventory-gst-calculation
```

9.2 Keeping Your Branch Up to Date

When dev advances while you are working on your feature branch (which it always does), you must bring those changes into your branch. Do this at least once a day.

Method A — Rebase (preferred — keeps clean linear history):

```
git fetch origin
```

```
git rebase origin/dev
```

If conflicts occur during rebase:

1. Resolve conflicts in the marked files

2. git add <resolved-file>

3. git rebase --continue

4. If stuck: git rebase --abort (returns to before rebase)

Method B — Merge (simpler but creates merge commits):

```
git fetch origin
git merge origin/dev

# After rebase, you must force-push (ONLY safe on YOUR feature branch):
git push --force-with-lease origin feature/your-branch
# --force-with-lease is safer than --force: fails if someone else pushed
```

 **REBASE VS MERGE:** Rebase rewrites your commits on top of the latest dev, creating a clean linear history. Merge creates a merge commit that shows the branches joined. ThunderERP prefers rebase for feature branches — but NEVER rebase dev or main.

10. Pull Requests — The Code Review Gate

A Pull Request (PR) is the formal process for merging your feature branch into dev. It is not just a merge request — it is a structured code review, an automated test run, and a knowledge-sharing opportunity.

10.1 Before Opening a PR — Self-Review Checklist

- Did you pull the latest dev and resolve any conflicts before pushing?
- Did you run `npm run lint` locally and fix all issues?
- Did you run `npm test` locally and pass all tests?
- Did you run `npm run build` and confirm it compiles?
- If you changed `schema.prisma`, did you run `npx prisma generate`?
- Is every commit message following Conventional Commits format?
- Is the PR under 300 lines of code change? If not, can it be split?
- Did you add or update tests for any new logic?
- Did you check the Swagger output if you added/changed API endpoints?

10.2 The PR Template

```
## Summary
Brief description of what this PR does and why.

## Changes
- Added GST calculation method to InventoryService
- Added GST percentage field to CreateProductDto
- Added unit tests for GST edge cases (0%, 5%, 12%, 18%, 28%)

## Type of Change
- [x] New feature (non-breaking)
- [ ] Bug fix
- [ ] Breaking change
- [ ] Database migration included

## Testing
- [x] npm run lint - passes
- [x] npm test - all tests pass
- [x] npm run build - compiles
- [x] Tested in Postman with real API calls

## Screenshots (UI changes only)
<!-- Attach before/after screenshots for any frontend changes -->

## Checklist
- [x] Branch created from latest dev
- [x] Conventional commit messages used
- [x] No secrets or .env files committed
- [x] Self-reviewed before opening PR
```

10.3 PR Size Rules

PR Size	Lines Changed	Reviewer Time	Action
Ideal	Under 200 lines	15–30 minutes	Proceed normally
Acceptable	200–400 lines	30–60 minutes	Add extra context in description
Too large	Over 400 lines	1+ hours	MUST split into smaller PRs
Emergency hotfix	Any size	Immediate	Flag as hotfix, Tech Lead fast-tracks review

10.4 Code Review — What Reviewers Check

Review Area	Specific Questions
ERP Logic Correctness	Does this follow ThunderERP business rules? Does stock deduction use a transaction?
TypeScript Safety	Is "any" used anywhere? Are all types explicit and correct?
Database Integrity	Are multi-table operations wrapped in <code>prisma.\$transaction()</code> ?
Soft Delete	Does the code use <code>isActive: false</code> instead of <code>DELETE</code> ?
Audit Trails	Does every important change write to <code>AuditLog</code> ?
Security	Is user input validated via DTO? Are auth guards applied?
Error Handling	Are correct HTTP exceptions thrown? Is every error state handled?
Test Coverage	Are new service methods tested? Are edge cases covered?
Performance	Are there N+1 query problems? Are indexes used for heavy queries?

10.5 Responding to Review Comments

- Acknowledge every comment — even if you disagree, respond before pushing.
- Fix issues in new commits — do not amend old commits on an open PR (makes diff hard to follow).
- Resolve conversations — click "Resolve conversation" after addressing a comment.
- Request re-review explicitly — after pushing fixes, click "Re-request review" on the reviewer.
- Disagreements — discuss politely in the comment thread. Tech Lead has final say.

10.6 CODEOWNERS — Automatic Review Assignment

```
# .github/CODEOWNERS
# These people are automatically assigned as reviewers:

# Tech Lead reviews everything:
* @tech-lead-github-username

# Module leads for specific modules:
src/modules/finance/    @finance-lead @tech-lead-github-username
src/modules/inventory/  @inventory-lead @tech-lead-github-username
src/modules/auth/       @security-lead @tech-lead-github-username

# Database changes require extra review:
prisma/                 @dba-lead @tech-lead-github-username

# CI/CD changes are critical:
.github/                @devops-lead @tech-lead-github-username
```

11. Resolving Merge Conflicts

Merge conflicts happen when two branches have changed the same lines of the same file. They are not errors — they are Git asking you to make a decision about which version to keep.

11.1 How Conflicts Look

```
// Git inserts these markers into the conflicted file:

<<<<<<< HEAD (your branch)
const gstAmount = product.price * (dto.gstPercentage / 100);
=====
const gstAmount = product.basePrice * (product.gstRate / 100);
>>>>>>> origin/dev

// Meaning:
// Between <<<<<<< and ===== : YOUR version (current branch)
// Between ===== and >>>>>>> : THEIR version (what you are merging in)

// Resolution: Edit the file to keep the correct version:
const gstAmount = product.price * (dto.gstPercentage / 100);
// Delete all the <<<<<<< ===== >>>>>>> markers completely.
```

11.2 Step-by-Step Conflict Resolution

12. Run git status to see which files have conflicts.
13. Open each conflicted file. Search for <<<<<<< to find all conflict markers.
14. Understand BOTH versions before deciding — read the context around each change.
15. Edit the file to keep the correct version (might be yours, theirs, or a combination).
16. Remove ALL conflict markers: <<<<<<<, =====, >>>>>>>.
17. Run the tests to verify the resolution is correct.
18. Stage the resolved file: git add <filename>.
19. Continue the rebase or commit the merge: git rebase --continue or git commit.

11.3 Conflict Prevention

- Pull from dev every morning before starting work.
- Keep PRs small — large PRs conflict with more things.
- Communicate in Slack when you are working on a heavily shared file (e.g., app.module.ts).
- Merge your PR quickly after approval — the longer it sits, the more it diverges from dev.

11.4 Resolving Conflicts in GitHub Desktop

GitHub Desktop provides a visual conflict resolver. Open the conflicted repository in Desktop → it will highlight conflicts and let you choose "Use mine", "Use theirs", or open in VS Code for manual resolution.



WHEN IN DOUBT: Do NOT guess at a conflict resolution for financial or inventory logic. Call the other developer, understand both sides, and resolve together. A wrong resolution in the order creation service can corrupt financial data.

12. Enhanced Debugging Playbook

When something goes wrong with Git or CI, do not panic and do not guess. Follow this layered, structured approach. 90% of problems are solved in Layers 1–3.

Layer 1 — Immediate Context Check

```
# What branch am I on?
git branch
# Expected: * feature/inventory-gst-calculation

# What is the state of my working directory?
git status
# Are there uncommitted changes? Staged files?

# What are my recent commits?
git log --oneline -10

# Am I ahead or behind remote?
git status
# Shows: "Your branch is ahead/behind origin/... by X commits"
```

Layer 2 — Sync & Pull Issues

```
# Fetch latest without merging (safe — just checks):
git fetch origin
git status
# If behind: pull the latest
git pull origin dev

# If pull creates conflicts — see Section 11.

# If you accidentally started on the wrong branch:
git stash                # Save your work
git checkout -b feature/correct-branch # Create correct branch
git stash pop            # Restore your work
```

Layer 3 — CI/CD Pipeline Failure

```
# Go to GitHub → your PR → "Checks" tab
# Click the failed workflow → expand the failed step
# Read the exact error message

# Common CI failures and local reproduction:

# Lint failure:
npm run lint
# Fix the reported issues, then commit and push.
```

```
# Test failure:
npm test
# Or run just the failing test file:
npm test -- --testPathPattern=inventory.service.spec.ts

# Build failure:
npm run build
# TypeScript errors show here — fix and push.

# Prisma validation failure:
npx prisma generate
npx prisma validate
# Or migration issue:
npx prisma migrate dev

# Node version mismatch:
node -v
# Compare with .nvmrc or package.json engines field.
# Use nvm to switch: nvm use 20
```

Layer 4 — Merge / Conflict Problems

```
# Check which files have conflicts:
git status
# Look for: "both modified: src/..."

# Open each file and resolve markers (see Section 11)

# After resolving all conflicts:
git add .
git rebase --continue # If in a rebase
# OR:
git commit           # If in a merge
git push origin feature/your-branch
```

Layer 5 — "Code Disappeared" or Wrong State

```
# The reflog — shows EVERY action you have ever taken:
git reflog

# Output (most recent first):
# 3a7f2b1 HEAD@{0}: commit: feat(inventory): add GST calculation
# 8c3e1d9 HEAD@{1}: checkout: moving from dev to feature/inventory-gst
# f2a9b4c HEAD@{2}: pull: Fast-forward

# If you lost a commit — find its hash in reflog, then recover:
git cherry-pick 3a7f2b1

# If you reset too far back:
```

```
git reflog
# Find where HEAD was before the reset
git reset --hard HEAD@{3}    # Go back to that state

# If you accidentally deleted a branch:
git reflog | grep "feature/deleted-branch"
git checkout -b feature/recovered-branch <sha-from-reflog>
```

Layer 6 — Deep Recovery

```
# Undo last commit, keep changes (local only - before pushing):
git reset --soft HEAD~1

# Safely undo a pushed commit on a shared branch:
git revert <commit-sha>
# This creates a NEW commit that undoes the changes.
# Safe because it does not rewrite history.

# Nuclear option - wipe local and match remote exactly:
# WARNING: ALL uncommitted and unpushed work is LOST
git reset --hard origin/dev

# If your local dev is in a mess - clean slate:
git stash          # Save anything you want to keep
git checkout dev
git reset --hard origin/dev
git stash pop      # Restore your stashed work
```

Problem	Solution
Committed to wrong branch	git log → get sha → git checkout correct-branch → git cherry-pick <sha> → git checkout wrong-branch → git reset --soft HEAD~1
Pushed to wrong branch	Open a PR from that branch. Reviewer will redirect. Or git revert on the wrong branch.
Merge conflict during PR	git fetch → git rebase origin/dev → resolve conflicts → git push --force-with-lease
CI passing locally but failing in GitHub	Check Node version (.nvmrc). Check if .env vars are set in GitHub Secrets. Run CI step commands exactly.
PR shows hundreds of unrelated changes	Your branch diverged too far from dev. git rebase origin/dev or re-open from latest dev.
"Non-fast-forward" push error	Someone pushed to the same branch. git pull → resolve → git push.
Accidentally staged .env file	git restore --staged .env → add .env to .gitignore → confirm it is not in commit

13. GitHub Actions — CI/CD Pipelines

GitHub Actions is the automation engine behind ThunderERP's quality gates. Every push to a pull request and every merge to dev or main triggers a set of automated workflows. These are not optional checks — they are hard gates that block broken code from reaching production.

13.1 Workflow Files Location

```
.github/
└─ workflows/
    ├── pr-checks.yml           ← Runs on every PR push
    ├── staging-deploy.yml      ← Runs on merge to dev
    └── production-deploy.yml  ← Runs on merge to main
```

13.2 PR Checks Workflow

```
# .github/workflows/pr-checks.yml
name: PR Checks

on:
  pull_request:
    branches: [dev, main]

jobs:
  lint:
    name: ESLint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20", cache: "npm" }
      - run: npm ci
      - run: npm run lint

  type-check:
    name: TypeScript
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20", cache: "npm" }
      - run: npm ci
      - run: npx tsc --noEmit

  test:
    name: Unit + Integration Tests
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
```

```
env:
  POSTGRES_PASSWORD: testpassword
  POSTGRES_DB: thunder_erp_test
options: >-
  --health-cmd pg_isready
  --health-interval 10s
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with: { node-version: "20", cache: "npm" }
  - run: npm ci
  - run: npx prisma generate
  - run: npx prisma migrate deploy
  env:
    DATABASE_URL:
postgresql://postgres:testpassword@localhost:5432/thunder_erp_test
  - run: npm test
  env:
    DATABASE_URL:
postgresql://postgres:testpassword@localhost:5432/thunder_erp_test

build:
  name: Build Verification
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: "20", cache: "npm" }
    - run: npm ci
    - run: npm run build

prisma-validate:
  name: Prisma Schema Validation
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: "20", cache: "npm" }
    - run: npm ci
    - run: npx prisma validate
    - run: npx prisma generate

security:
  name: Dependency Vulnerability Scan
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm audit --audit-level=high
```

13.3 Staging Deployment Workflow

```
# .github/workflows/staging-deploy.yml
name: Deploy to Staging

on:
  push:
    branches: [dev]

jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t thundererp-backend:staging .

      - name: Push to registry
        run: |
          echo ${ secrets.DOCKER_PASSWORD } | docker login -u ${ secrets.DOCKER_USERNAME } --password-stdin
          docker push thundererp-backend:staging

      - name: Deploy to staging server
        uses: appleboy/ssh-action@master
        with:
          host: ${ secrets.STAGING_HOST }
          username: ${ secrets.STAGING_USER }
          key: ${ secrets.STAGING_SSH_KEY }
          script: |
            cd /app/thunder-erp
            docker pull thundererp-backend:staging
            npx prisma migrate deploy
            docker compose up -d --no-deps backend

      - name: Run smoke tests
        run: |
          sleep 15 # Wait for app to start
          curl -f https://staging.thundererp.com/health || exit 1

      - name: Notify Slack
        uses: 8398a7/action-slack@v3
        with:
          status: ${ job.status }
          text: "Staging deployment: ${ job.status }"
        env:
          SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }
```

13.4 Production Deployment Workflow

```
# .github/workflows/production-deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment: production # Requires manual approval gate
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - name: Get version from package.json
        id: version
        run: echo "VERSION=$(node -p "require('./package.json').version")" >>
$GITHUB_OUTPUT

      - name: Build production Docker image
        run: docker build -t thundererp-backend:${{ steps.version.outputs.VERSION
}} .

      - name: Push production image
        run: docker push thundererp-backend:${{ steps.version.outputs.VERSION }}

      - name: Zero-downtime deploy
        # Blue-green or rolling update strategy
        uses: appleboy/ssh-action@master
        with:
          script: |
            cd /app/thunder-erp
            docker pull thundererp-backend:${{ steps.version.outputs.VERSION }}
            npx prisma migrate deploy # Run migrations first
            docker compose up -d --no-deps --scale backend=2 backend
            sleep 10
            docker compose up -d --no-deps --scale backend=1 backend

      - name: Create GitHub Release
        uses: actions/create-release@v1
        with:
          tag_name: v${{ steps.version.outputs.VERSION }}
          release_name: Release v${{ steps.version.outputs.VERSION }}
          generate_release_notes: true
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

13.5 GitHub Secrets — Storing Credentials Safely

GitHub Secrets store sensitive values that workflows need — database passwords, API keys, deploy credentials. They are encrypted and never visible in logs.

```
# Add secrets in GitHub:
# Repository → Settings → Secrets and variables → Actions → New secret

# Required secrets for ThunderERP:
DATABASE_URL           ← Production DB connection string
STAGING_DATABASE_URL   ← Staging DB connection string
JWT_SECRET             ← JWT signing secret
DOCKER_USERNAME        ← Docker registry username
DOCKER_PASSWORD        ← Docker registry password / token
STAGING_HOST           ← Staging server IP/hostname
STAGING_USER           ← SSH username for staging server
STAGING_SSH_KEY        ← Private SSH key for staging server
PRODUCTION_SSH_KEY     ← Private SSH key for production server
SLACK_WEBHOOK          ← Slack webhook URL for notifications

# Access in workflow:
${{ secrets.JWT_SECRET }}
```

 **SECRETS RULE:** Never put real credentials in workflow YAML files, even temporarily. Never echo or print secrets in workflow steps. GitHub masks known secrets in logs, but do not rely on that as a safety net.

14. Git Tags & Releases — Versioning Production

Every production deployment in ThunderERP is versioned with a Git tag. Tags are permanent, immutable markers in the commit history. They allow instant rollbacks, clear changelogs, and traceability of what is running in production.

14.1 Semantic Versioning

ThunderERP follows Semantic Versioning (SemVer): MAJOR.MINOR.PATCH

Version Part	When to Increment & Examples
MAJOR (v2.0.0)	Breaking changes — API changes that break existing integrations, major schema migrations.
MINOR (v1.3.0)	New features that are backwards-compatible — new endpoints, new modules.
PATCH (v1.2.5)	Bug fixes that are backwards-compatible — hotfixes, small corrections.

14.2 Tagging Commands

```
# Create an annotated tag (always use annotated, not lightweight):
git tag -a v1.3.0 -m "Release v1.3.0: Add GST calculation and partial payment"

# Push a specific tag:
git push origin v1.3.0

# Push all tags at once:
git push origin --tags

# List all tags:
git tag
git tag --list "v1.*"      # Filter by pattern

# See tag details:
git show v1.3.0

# Delete a tag (use only if created by mistake — before push):
git tag -d v1.3.0
git push origin --delete v1.3.0    # Delete from remote

# Checkout code at a specific tag (for inspection or hotfix):
git checkout v1.2.5
# WARNING: This puts you in "detached HEAD" state.
# To create a hotfix from a tag:
git checkout -b hotfix/v1.2.6-fix-order-total v1.2.5
```

14.3 GitHub Releases

GitHub Releases are a layer on top of Git tags. They add release notes, downloadable assets, and a changelog. The production deployment workflow creates these automatically. Tech Leads can also create them manually on GitHub → Releases → Draft a new release.

15. Advanced Git Techniques

15.1 Cherry-Pick — Apply a Specific Commit

Cherry-pick takes a single commit from any branch and applies it to your current branch. Useful for applying a hotfix from main back to dev, or recovering a commit from a deleted branch.

```
# Apply a specific commit to your current branch:
git cherry-pick 3a7f2b1

# Apply multiple commits:
git cherry-pick 3a7f2b1 8c3e1d9 f2a9b4c

# Apply a range of commits:
git cherry-pick 3a7f2b1..f2a9b4c

# Cherry-pick without auto-committing (stage only):
git cherry-pick --no-commit 3a7f2b1

# If conflicts occur during cherry-pick:
# Resolve conflicts → git add . → git cherry-pick --continue
# To abort: git cherry-pick --abort
```

15.2 Interactive Rebase — Rewrite Local History

Interactive rebase lets you reorder, squash, rename, or delete commits BEFORE they are pushed. This is how you clean up a messy commit history before opening a PR.

⚠ RULE: ONLY use interactive rebase on commits that have NOT been pushed yet. Never rebase commits that other people have pulled — it rewrites history and breaks their copies.

```
# Interactively rebase your last 4 commits:
git rebase -i HEAD~4

# Git opens an editor with:
pick 3a7f2b1 feat(inventory): add product model
pick 8c3e1d9 WIP: half done GST
pick f2a9b4c fix typo
pick 9d8e7f6 feat(inventory): finish GST calculation

# Change "pick" to:
# squash (s)  - merge into previous commit
# reword (r)  - keep commit but edit message
# drop (d)    - delete this commit entirely
# fixup (f)   - squash into previous, discard this message

# Result after squashing WIP + fix typo + finish into the first:
```

```
pick 3a7f2b1 feat(inventory): add product model
squash 8c3e1d9 WIP: half done GST
squash f2a9b4c fix typo
squash 9d8e7f6 feat(inventory): finish GST calculation

# Git merges them and asks you to write one clean message:
feat(inventory): add product model with GST calculation
```

15.3 Git Bisect — Find Which Commit Broke Something

Git bisect performs a binary search through your commit history to find the exact commit that introduced a bug.

```
# Start bisect:
git bisect start

# Mark current state as bad (bug exists here):
git bisect bad

# Mark a known good commit (bug did not exist here):
git bisect good v1.2.0

# Git checks out a commit in the middle.
# Test if the bug exists. Then tell Git:
git bisect good    # No bug here
# OR
git bisect bad     # Bug exists here

# Git keeps narrowing until it finds the exact commit.

# When done:
git bisect reset   # Return to HEAD
```

15.4 Git Submodules & Worktrees

```
# Git Worktree — check out two branches simultaneously:
# Useful when you need to test main while working on a feature
git worktree add ../thunder-erp-main main
# Now ../thunder-erp-main has main branch checked out
# Your original folder still has feature branch

# Remove worktree when done:
git worktree remove ../thunder-erp-main

# Git Submodules — include another repo inside your repo:
# Used for shared libraries across backend and frontend
git submodule add git@github.com:thundererp/thunder-erp-shared.git shared
git submodule update --init --recursive
```

16. Security & Best Practices

16.1 Never Commit Secrets — The Rules

Never Commit	Correct Alternative
.env files with real values	Commit .env.example (template). Real values in GitHub Secrets.
Database connection strings	GitHub Secrets → DATABASE_URL. Read from process.env in code.
JWT secrets	GitHub Secrets → JWT_SECRET.
SSH private keys	Add to SSH agent locally. Add public key to GitHub. Never in code.
API keys (Stripe, Twilio, AWS)	GitHub Secrets. Read via process.env. Rotate immediately if leaked.
Passwords in code comments	Remove them. Use a password manager for team credentials.

16.2 Branch Protection Rules — GitHub Settings

These rules are configured in GitHub → Repository → Settings → Branches. They apply to dev and main and cannot be overridden by anyone.

- Require a pull request before merging — no direct pushes ever.
- Require at least 1 approving review before merging.
- Dismiss stale PR approvals when new commits are pushed — prevents sneaking in changes after approval.
- Require status checks to pass (CI must pass) before merging.
- Require branches to be up to date before merging — prevents stale branch merges.
- Restrict who can push to main — Tech Leads only.
- Require signed commits (advanced) — verifies commit author identity.

16.3 CODEOWNERS for ERP-Critical Modules

Finance, inventory, and auth modules handle sensitive business logic. Any PR touching these requires review from the designated module lead and the Tech Lead, enforced via CODEOWNERS.

16.4 Dependency Security

```
# Audit dependencies for known vulnerabilities:
npm audit

# Audit with exit code (for CI - fails if high severity found):
npm audit --audit-level=high

# Auto-fix safe updates:
npm audit fix
```

```
# Review what will change before fixing:
npm audit fix --dry-run

# Check for outdated packages:
npm outdated

# Dependabot (enable in GitHub):
# GitHub → Settings → Security → Dependabot alerts
# Creates automatic PRs when dependencies have vulnerabilities.
```

17. Docker in the Development Workflow

Docker ensures every developer and every CI pipeline runs the same versions of PostgreSQL, Node.js, and the application itself. "Works on my machine" is eliminated when everyone uses Docker.

17.1 docker-compose.yml — Local Development

```
# docker-compose.yml - commit this to git
version: "3.8"

services:
  postgres:
    image: postgres:16-alpine
    container_name: thunder-postgres
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped

  backend:
    build: .
    container_name: thunder-backend
    depends_on: [postgres]
    environment:
      DATABASE_URL: ${DATABASE_URL}
      JWT_SECRET: ${JWT_SECRET}
    ports:
      - "3000:3000"
    volumes:
      - ./src:/app/src # Hot-reload in development
    command: npm run start:dev

volumes:
  postgres_data:
```

17.2 Dockerfile — Production Image

```
# Dockerfile - multi-stage build for smaller production image

# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate
```

```
RUN npm run build

# Stage 2: Production
FROM node:20-alpine AS production
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma
COPY prisma ./prisma

EXPOSE 3000
CMD ["node", "dist/main.js"]
```

17.3 Docker Commands Reference

Command	Purpose
docker compose up -d	Start all services in background
docker compose down	Stop all services (data preserved)
docker compose down -v	Stop + DELETE all data (volumes)
docker compose logs -f backend	Follow backend logs in real time
docker compose restart backend	Restart just the backend service
docker compose exec backend sh	Open shell inside running container
docker ps	List running containers
docker images	List local Docker images
docker system prune	Remove unused containers, images, networks

18. Team Collaboration Rules

With 35+ developers on a shared modular monolith, clear communication rules prevent the majority of merge conflicts, duplicated work, and team friction.

18.1 Communication Rules

Situation	Required Action
Starting work on a heavily shared file (app.module.ts, schema.prisma)	Post in Slack: "@team working on app.module.ts for auth changes — will PR by EOD"
Your PR will be large (300+ lines)	Warn your reviewer in advance. Consider splitting into smaller PRs.
You find a bug that affects another module	Create a GitHub Issue. Tag the module lead. Do not fix it in your current PR.
A production issue is found	Immediately notify Tech Lead via direct message. Do not silently hotfix.
You are blocked by another PR	Comment on that PR explaining your dependency. Tag the reviewer to expedite.
You are going to be unavailable	Post in Slack with open PRs that need attention. Assign a reviewer explicitly.

18.2 GitHub Issues — Tracking Work

Every feature, bug, and task should have a GitHub Issue before a branch is created. Issues provide context, history, and a paper trail for every decision.

```
# Issue title format:
[Module] Brief description of the task or bug

# Good issue titles:
[Inventory] Add GST percentage field to product creation
[Finance] Bug: Invoice total includes cancelled order items
[Auth] Implement refresh token rotation

# Linking branch to issue:
# Branch: feature/inventory-gst-calculation
# In the PR description: "Closes #143"
# GitHub automatically closes issue #143 when the PR merges.

# Issue labels used in ThunderERP:
# bug          → Something is broken
# feature      → New capability
# priority:high → Blocking other work or production issue
# module:inventory, module:sales, module:finance, etc.
```

18.3 GitHub Projects — Sprint Board

ThunderERP uses GitHub Projects as a Kanban board to track sprint work. Every issue is on the board.
Columns: Backlog → In Progress → In Review → Done.

- Move your issue to "In Progress" when you start a branch.
- Move to "In Review" when you open a PR.
- Move to "Done" only after the PR is merged to dev.
- Never have more than 2 items "In Progress" simultaneously.

19. Integration with ThunderERP Development Workflow

19.1 NestJS-Specific Git Rules

Rule	Implementation
Prisma schema changes always include migration	Any PR that changes schema.prisma must include a migration file in prisma/migrations/
Run npx prisma generate before pushing	If you changed schema.prisma, the generated client must be regenerated. CI checks this.
Never commit generated Prisma client	The prisma/generated/ folder is in .gitignore — it is regenerated in CI and on each machine.
NestJS module registrations in separate commit	Adding a new module to app.module.ts is a separate commit from the module implementation.
API changes update Swagger	If you add/change an endpoint, verify Swagger at /api shows the correct docs before PR.

19.2 React-Specific Git Rules

Rule	Implementation
npm run build must pass	TypeScript errors prevent the build. CI enforces this. Fix all type errors before pushing.
Screenshots required for UI changes	Every PR that changes visible UI must attach before/after screenshots in the PR description.
Bundle size check	If you add a new dependency, mention it in the PR description. Avoid large libraries.
Environment variable changes	Update .env.example whenever you add a new VITE_ variable. Never add real values.

19.3 Database-Specific Git Rules

Rule	Implementation
Never edit migration files after pushing	Migration files are immutable once committed. Create a new migration to fix issues.
Migration name must be descriptive	npx prisma migrate dev --name add_gst_percentage_to_products — not "update" or "fix".
Test migration locally before PR	Run npx prisma migrate dev on your local DB. Verify data is not corrupted.
Include rollback plan for breaking migrations	PR description must document how to roll back a breaking schema change.

20. Common Mistakes — Strictly Avoid

Mistake	Why It Is Dangerous & Correct Approach
Not pulling before starting work	Your branch starts from stale code. Conflicts accumulate throughout the day. Always: <code>git checkout dev</code> → <code>git pull origin dev</code> → create branch.
Committing directly to dev or main	Bypasses CI checks and code review. Can break production instantly. NEVER. Always use a feature branch and PR.
Ignoring CI failures and asking for review anyway	Wastes reviewers' time. Broken code goes to the review queue. Fix CI first, then request review.
Large PRs with 1000+ line changes	No one can review 1000 lines thoroughly. Bugs hide in size. Split into multiple focused PRs under 300 lines each.
Vague commit messages like "fix" or "update"	Makes git log useless for debugging. Two months from now, no one knows what "fix" did. Use Conventional Commits always.
Committing .env files	Exposes database passwords, JWT secrets, and API keys permanently in git history. Rotate all secrets immediately if this happens.
Using <code>git reset --hard</code> on dev or main	Rewrites shared history. Destroys other developers' commits. Use <code>git revert</code> instead.
Force pushing to a shared branch	<code>git push --force</code> on dev or main overwrites others' work permanently. Forbidden. Use <code>--force-with-lease</code> only on your own feature branches.
Not cleaning up branches after merge	Hundreds of stale branches pollute the repository. Delete feature branches after PR merges.
Working on a stale branch for days without syncing	Conflicts compound daily. Sync with dev via rebase every morning.
Not reading CI error logs before asking for help	The error is always in the logs. Read the exact failure message. 90% of CI issues are self-diagnosable.
Skipping the PR template	Reviewers lack context. Reviews take longer. Fill in every section of the PR template.

21. Quick Reference Cheat Sheet

Daily Commands

Task	Command
Start of day — sync with dev	git checkout dev && git pull origin dev
Create feature branch	git checkout -b feature/<module>-<description>
Check working directory state	git status
See recent commits	git log --oneline -10
Stage all changes	git add .
Stage specific file	git add src/modules/inventory/inventory.service.ts
Commit with Conventional format	git commit -m "feat(inventory): add GST calculation"
Push branch for first time	git push -u origin feature/inventory-gst-calculation
Push subsequent commits	git push
Update branch from dev (rebase)	git fetch origin && git rebase origin/dev
Save work temporarily	git stash && git stash pop
After PR merge — cleanup	git checkout dev && git pull && git branch -d feature/...

Recovery Commands

Situation	Command
Unstage a file	git restore --staged <file>
Discard working dir changes	git restore <file>
Undo last commit (before push)	git reset --soft HEAD~1
Safely undo a pushed commit	git revert <commit-sha>
Find lost commits	git reflog
Recover lost commit	git cherry-pick <sha-from-reflog>
See all branches	git branch -a
See differences between branches	git diff main..dev --name-only
Find which commit broke something	git bisect start → git bisect good/bad
Clean local to match remote	git reset --hard origin/dev (DANGER — loses local work)

CI/CD Quick Fixes

CI Failure	Local Fix Command
Lint failure	npm run lint (fix errors, commit, push)
TypeScript failure	npx tsc --noEmit (fix errors, commit, push)
Test failure	npm test (or npm test -- --testPathPattern=<file>)
Build failure	npm run build (fix TypeScript errors)
Prisma validation failure	npx prisma validate && npx prisma generate
Migration failure	npx prisma migrate dev (check migration file, commit)
Security audit failure	npm audit fix --force (review changes carefully first)
Node version mismatch	nvm use 20 (or version in .nvmrc)

Conventional Commit Quick Reference

Type	Example
feat	feat(inventory): add product barcode scanning support
fix	fix(sales): resolve race condition in stock deduction
refactor	refactor(auth): extract token validation to utility function
docs	docs(readme): update local setup instructions for Docker
test	test(finance): add integration tests for invoice payment flow
chore	chore(deps): update Prisma from 5.8 to 5.12
style	style(inventory): reorder imports per ESLint rules
perf	perf(products): add composite index on tenantId and status
ci	ci: add Node 20 version constraint to all workflows

22. Golden Rules of ThunderERP Git & GitHub

RULE 1: Always pull from dev before starting any work. Your branch must start from the latest state.
`git checkout dev → git pull origin dev → create branch.`

RULE 2: Never commit directly to dev or main. Every change — no matter how small — goes through a feature branch and a Pull Request. No exceptions.

RULE 3: Follow THE COMPLETE GIT + CI FLOW (Section 7) for every single change. It is not a suggestion. It is the safety net for the entire ERP.

RULE 4: CI must pass before requesting code review. A failed CI is broken code. Fix it first, then ask for review. Never ask a reviewer to look at red checks.

RULE 5: Write atomic commits with Conventional Commit messages. One commit, one change, one clear reason. "fix" or "update" are not commit messages.

RULE 6: Communicate early when working on shared modules. Announce in Slack before touching `app.module.ts`, `schema.prisma`, or any file two people own.

RULE 7: Never commit secrets. No `.env` files, no passwords, no API keys, no JWT secrets. Ever. If it happens accidentally, rotate the secret immediately.

RULE 8: Main must always be deployable. If something breaks main, it is the highest priority issue for the entire team until it is fixed.

RULE 9: Keep PRs small. Under 300 lines. Reviewers cannot effectively review 1000-line PRs. Split your work. Small PRs merge faster and have fewer bugs.

RULE 10: Git + CI/CD is not just tooling. It is a team coordination system, a safety net, and the permanent history of ThunderERP. Treat it with the same care as the database.

23. Learning & Progression Roadmap

Week 1 — Foundation

- Complete setup: Git, GitHub account, SSH key, repository clone (Section 2).
- Read and understand the mental model: four areas of Git (Section 3).
- Practice daily lifecycle: pull dev, create branch, commit, push (Section 9).
- Open your first pull request to dev with a small feature.
- Read the Conventional Commits specification.

Month 1 — Daily Competency

- Daily workflow is automatic muscle memory — no need to look up commands.
- Conventional Commit messages on every commit, every time.
- Can read and understand CI pipeline output when a check fails.
- Can resolve simple merge conflicts independently.
- PRs are consistently under 300 lines with clear descriptions.

Month 2 — Intermediate Proficiency

- Can diagnose and fix any CI failure by reading the error log.
- Understands rebase vs merge and uses the right one in each situation.
- Can use git stash, git cherry-pick, and git reflog confidently.
- Participates as a code reviewer — can identify ERP logic issues in PRs.
- Understands GitHub Actions workflow structure — can read and debug YAML.

Month 3+ — Advanced Level

- Can perform emergency recovery using git reflog and git reset.
- Can write and modify GitHub Actions workflows.
- Understands semantic versioning and the release tagging process.
- Can perform an interactive rebase to clean commit history.
- Acts as a technical mentor for newer team members on Git workflow.
- Understands Docker integration in the CI/CD pipeline end to end.

24. Final Mindset — What You Are Really Protecting

Git + GitHub + CI/CD is not just a set of tools. It is a team coordination system, a safety net, and the permanent, unalterable history of ThunderERP. Every commit you make, every PR you open, every CI check you pass or ignore — these decisions have real consequences for real users and real data.

What Poor Git Discipline Can Cause

```
A direct push to main without tests
→ Broken production for 500 active users
→ Orders fail to create
→ Revenue loss and client trust damage

A committed .env file with JWT_SECRET
→ Anyone with GitHub access can forge auth tokens
→ Unauthorized access to all ERP data
→ GDPR violation and potential legal consequences

A large PR with a buried bug in line 847 of 1000
→ Reviewer misses the bug
→ Broken stock deduction ships to production
→ Inventory goes negative, customer orders oversold

Ignoring a CI failure "just this once"
→ The one unreviewed broken commit
→ Failed Prisma migration on production deploy
→ Database schema inconsistency — system offline
```

What Excellent Git Discipline Enables

- Any developer can join the team and be productive in their first week.
- Any bug can be traced to the exact commit, author, and reason it was introduced.
- Any deployment can be rolled back to any previous version in minutes.
- The entire history of every decision in ThunderERP is preserved permanently.
- 35 developers can work simultaneously with confidence that their work is safe.

Mastering the Complete Git + CI Flow turns you from an individual coder into a reliable professional who protects the entire ERP system. That is the standard ThunderERP holds every engineer to — intern or senior.

Every commit is permanent. Make it count.